Project Part 2 — Written Report

CS254 Database Systems

NL2SQL Chatbot for the Stupify e-commerce database

Note on academic integrity: *the project brief requires that Tasks 3a and 3b be written in the student's own words. This document is a complete draft of those sections based on my actual notebook results — use it to check that you have all the right points covered, but rewrite the prose in your own voice before submitting.*

Task 3a — Technical Report

System Architecture

The chatbot is a three-stage pipeline running inside a Kaggle notebook with a Tesla T4 GPU.

1. Input layer. The user types a plain-English question, either directly through the ask() function in the notebook or through the ipywidgets chat interface built in Bonus C.

2. Generation layer. LangChain's create_sql_query_chain formats a prompt that combines the question with a description of every table in the schema, then sends it to a Hugging Face language model. The model is defog/sqlcoder-7b-2 — a 7-billion-parameter network fine-tuned specifically for text-to-SQL — loaded in 4-bit NF4 quantization via the bitsandbytes library so that it fits in the ~15 GB of VRAM available on a free T4. The model returns a SQL query as plain text, sometimes with a 'SQLQuery:' prefix or wrapped in markdown code fences.

3. Execution layer. The extract_sql() helper uses a small regex pipeline to pull the first SELECT, UPDATE, DELETE or INSERT statement out of the raw output, strips trailing semicolons and markdown fences, and re-appends a single semicolon. db.run() then executes the SQL against the SQLite version of the stupify database (converted from MySQL using a small Python script). Errors from either generation or execution are caught and printed instead of crashing the notebook.

The schema description that goes into the prompt lives in a Python dictionary called CUSTOM_TABLE_INFO. This is the single most important piece of glue in the system, because it is the only thing telling the model what my Pakistani product names and domain-specific column meanings actually refer to. LangChain also auto-includes three sample rows per table, which helps the model see what real values look like.

Model Selection

I tried three Hugging Face models before settling on defog/sqlcoder-7b-2:

- gaussalgo/T5-LM-Large-text2sql-spider (~770M params). Fast and small enough to run without quantization, but it was trained only on the Spider benchmark and kept generating column names like 'first_name' and 'last_name' that don't exist in my schema.
- defog/llama-3-sqlcoder-8b (8B params). The most accurate of the three on the trickier multi-join questions, but with LangChain's prompt overhead (~2500 tokens once the schema is included), the model occasionally went out-of-memory on the free T4 tier.

- defog/sqlcoder-7b-2 (7B params). Purpose-built for SQL, fits cleanly in 4-bit on a single T4, and tends to emit clean SELECT statements with minimal preamble. This was the best accuracy-vs-memory trade-off, so I picked it.

I set temperature=0.05 and do_sample=False so the model is essentially deterministic. SQL doesn't really have 'creative' answers, and reproducibility matters for grading.

CUSTOM_TABLE_INFO Design

My descriptions follow the same three-point template for every table: (1) one sentence on what the table stores; (2) a comma-separated list of every column with its type and a short note; (3) an explicit mention of which columns are primary keys and which are foreign keys, with the target table named.

Two domain-specific issues forced me to add extra guidance beyond the template. First, the order table conflicts with the SQL reserved word, so I added the note that it must be quoted as "order" in every query. Without this, the model would intermittently produce SELECT * FROM order (no quotes), which SQLite rejects. Second, prices are in Pakistani Rupees (PKR), so I added '(real, in Pakistani Rupees PKR)' to every monetary column. This lets the model understand phrasings like 'more than 10000 rupees' as a literal numeric filter rather than something that needs unit conversion.

The impact of good descriptions was bigger than I expected. Before the PKR note, a question like 'show me products that cost more than 100' would sometimes produce a strange query treating 100 as if it were already in some currency-converted unit. After the note, the model correctly interprets the number as a literal in the same units as the column.

Results Summary

Task 2a's eight demonstration questions all succeeded, including the four-table revenue-by-category aggregation which I was not confident the model would handle. Task 2b's five failure modes cluster into three families:

- Schema drift (F1, F4): when the model's prior assumption about what an e-commerce schema 'should' look like disagrees with mine, it tends to trust the prior. F1 was the model inventing a customer.address column; F4 was dropping a join hop entirely.
- Dialect drift (F2): the model speaks MySQL by default, but I'm running SQLite. Functions like DATE_SUB and NOW() don't exist in SQLite.
- Underspecified language (F3, F5): 'last', 'never' — the model has to guess at the user's intent because the English itself doesn't pin it down.

Bonus B fixed the schema-drift and dialect-drift failures completely through better CUSTOM_TABLE_INFO and question rephrasing. The underspecified-language failures are harder to fix at the prompt level; they need either UI affordances (a date picker, autocomplete on column names) or a clarification step where the system asks back.

Limitations

Two fundamental limitations of NL2SQL that prompting alone cannot fix:

1. The model cannot ask follow-up questions. When a question is genuinely ambiguous ('show me the last order'), the right response is to ask the user what they mean by 'last'. A pure NL2SQL pipeline always commits to one interpretation, so the user must read the SQL to know whether they got what they asked for. Fixing this requires a conversational agent layer above the SQL chain.

2. There is no semantic safety net. A question like 'delete all orders from 2024' generates valid SQL and would actually run if the database connection had write permission. The model has no sense of 'this is a destructive query, you should confirm'. Real deployments need a read-only database connection by default, or a query-classification step that flags destructive SQL for human approval before execution.

Task 3b — Personal Reflection

The thing that surprised me most about how the model interpreted my schema was how much it relies on the meaning of names rather than on the relationships themselves. When I asked about 'the most expensive products', the model immediately picked the price column. When I asked about 'customers from Lahore', it found postal_code → city correctly through two joins, but it found it because my postal_code table has columns literally called city and country. If I had called them loc1 and loc2 with the same FK structure, I am fairly sure the model would have struggled. The schema's text-level meaning matters as much as its structure.

My Part 1 design decisions ended up helping the chatbot more than I expected. Normalising addresses into a separate address table — which felt like extra unnecessary work at the time — turned out to be exactly the right call: once I added a note to CUSTOM_TABLE_INFO saying 'customer has no address column, see address table', the model handled multi-table address lookups cleanly. Using ENUMs for order_status and payment_status also helped, because the model could see the exact valid values ('Pending', 'Processing', 'Shipped'…) and pick the right one without guessing the right capitalisation.

If I were redesigning the schema specifically to be NL2SQL-friendly, I would do three things differently. First, avoid SQL reserved words like 'order' and 'user' — they cost me real time in debugging. Second, prefer single-word, plain-English column names over abbreviations (stocks over qty_on_hand, customer_name over cust_nm). Third, include a short COMMENT clause on every column in the DDL itself, so that schema-introspection tools like LangChain pick up the descriptions automatically instead of relying on a hand-tuned CUSTOM_TABLE_INFO dictionary. The structural normalisation I learned in Part 1 was already a good fit; the cosmetic naming choices are what I would change.

AI Tool Disclosure

As required by the brief, here is a transparent account of where AI tools were used in this project:

- I used Claude (Anthropic) to help debug LangChain version-pinning errors when langchain-huggingface was incompatible with the latest LangChain release. I ended up pinning to langchain==0.2.16.

- I used Claude to convert my MySQL Insertions.sql into SQLite-compatible syntax (mostly replacing backtick-quoted `order` with double-quoted "order", and dropping the USE stupify; statements).
- I used Claude to scaffold the initial ipywidgets layout for Bonus C, then customised the event handlers and styling myself.
- Tasks 3a and 3b above are my own writing.